

Arrays, Strings & Basic I/O

Prof. Ashwani Mishra

Assistant Professor
AIML Department

Introduction to Arrays

- An array is a fixed-size container that holds multiple values of the same type.
- All elements share one name; an index (starting at 0) identifies each element.
- Array length is set at creation and cannot grow or shrink at runtime.
- Arrays can store primitives (`int[]`, `double[]`) or references (`String[]`, `Student[]`).
- The `length` property returns the number of elements; valid indexes are 0 to `length-1`.
- Arrays are objects in Java and live on the heap even when they hold primitives.

Figure 3.1 Introduction to Arrays

Array Types in Java

Type	Declaration Example	Notes
1D int array	<code>int[] marks = new int[5];</code>	Fixed size, index 0 to length-1
1D with init	<code>int[] nums = {10, 20, 30};</code>	Size inferred from values
2D array	<code>int[][] grid = new int[3][4];</code>	Rows and columns
2D jagged	<code>int[][] jagged = new int[3][];</code>	Each row can differ in length
length property	<code>arr.length</code>	Number of elements (not a method)

Figure 3.1 Introduction to Arrays

One-Dimensional Arrays

- Declare and initialize with `int[] marks = {85, 72, 90, 68, 95}` for known values.
- Alternatively use `new int[5]` to create an array of five default-initialized elements.
- Access elements with `marks[i]`; accessing `marks[5]` when length is 5 throws `ArrayIndexOutOfBoundsException`.
- Use a for loop with index `i` from 0 to `marks.length - 1` to traverse all elements.
- The enhanced for loop (`for (int m : marks)`) reads each element without managing the index.
- Compute aggregates such as sum and average by iterating over the array elements.

Figure 3.2 One-Dimensional Array

Array1D.java

```
public class Array1D {  
    public static void main(String[] args) {  
        int[] marks = {85, 72, 90, 68, 95};  
  
        System.out.println("Length: " + marks.length);  
  
        for (int i = 0; i < marks.length; i++) {  
            System.out.println("marks[" + i + "] = " + marks[i]);  
        }  
  
        int sum = 0;  
        for (int m : marks) sum += m;  
        System.out.println("Average: " + (sum / marks.length));  
    }  
}
```

Figure 3.2 One-Dimensional Array

Array Types and Properties

Array Types in Java

Type	Declaration Example	Notes
1D int array	<code>int[] marks = new int[5];</code>	Fixed size, index 0 to length-1
1D with init	<code>int[] nums = {10, 20, 30};</code>	Size inferred from values
2D array	<code>int[][] grid = new int[3][4];</code>	Rows and columns
2D jagged	<code>int[][] jagged = new int[3][];</code>	Each row can differ in length
length property	<code>arr.length</code>	Number of elements (not a method)

Figure 3.3 Array Types Reference

Two-Dimensional Arrays

- A 2D array is an array of arrays, useful for matrices, tables, and grids.
- Declare with `int[][]` matrix where each row can have the same or different column counts.
- Initialize with nested braces: `{{1,2,3}, {4,5,6}}` for a 2x3 matrix.
- Access element at row `r`, column `c` with `matrix[r][c]`.
- Nested for loops iterate rows outer and columns inner to print the full grid.
- Jagged arrays assign different row lengths after creating `int[][]` jagged = new `int[3][]`.

Figure 3.4 Two-Dimensional Array

Array2D.java

```
public class Array2D {  
    public static void main(String[] args) {  
        int[][] matrix = {  
            {1, 2, 3},  
            {4, 5, 6},  
            {7, 8, 9}  
        };  
  
        for (int row = 0; row < matrix.length; row++) {  
            for (int col = 0; col < matrix[row].length; col++) {  
                System.out.print(matrix[row][col] + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

Figure 3.4 Two-Dimensional Array

Enhanced for Loop with Arrays

- The for-each loop syntax is for (Type item : arrayName) and simplifies iteration.
- It eliminates manual index management and reduces off-by-one errors in loops.
- You can read each element but cannot modify the array index or replace elements by reference.
- for-each works with any array type and with collections that implement Iterable.
- Use a traditional for loop when you need the index or must modify elements in place.
- Enhanced for loops make code cleaner when processing every element sequentially.

Figure 3.5 Enhanced for Loop

ForEachArray.java

```
public class ForEachArray {  
    public static void main(String[] args) {  
        String[] subjects = {"OOPJ", "EP", "DSA", "DBMS"};  
  
        // Enhanced for loop (for-each)  
        for (String subject : subjects) {  
            System.out.println("Subject: " + subject);  
        }  
  
        // Works with any array or Iterable collection  
    }  
}
```

Figure 3.5 Enhanced for Loop

String Immutability

String Immutability

String (Immutable)

- Once created, content cannot change
- Every modification creates a new String object
 - Safe for sharing across threads
 - Use for constants and short text

StringBuilder (Mutable)

- Internal buffer can be modified in place
- append(), insert(), delete() change same object
- Efficient for loops and many concatenations
 - Not thread-safe unless synchronized

Figure 3.6 String Immutability

Understanding Strings in Java

- String is a class in java.lang used to represent sequences of characters (text).
- String objects are immutable: once created, their character sequence cannot change.
- Operations like toUpperCase() and concat() return new String objects, not modified originals.
- The string literal pool stores unique literals for memory efficiency and sharing.
- Use equals() to compare string content; == compares reference addresses only.
- Immutability makes strings thread-safe and suitable for constants and map keys.

Figure 3.7 String Concepts

String Immutability

String (Immutable)

- Once created, content cannot change
- Every modification creates a new String object
 - Safe for sharing across threads
 - Use for constants and short text

StringBuilder (Mutable)

- Internal buffer can be modified in place
- append(), insert(), delete() change same object
- Efficient for loops and many concatenations
 - Not thread-safe unless synchronized

Figure 3.7 String Concepts

Common String Methods

Common String Methods

Method	Returns	Example
length()	int length	"java".length() returns 4
charAt(i)	char at index	"java".charAt(0) returns 'j'
substring(a,b)	portion of string	"Hello".substring(1,3) returns "el"
equals(s)	boolean comparison	s1.equals(s2) compares content
toUpperCase()	uppercase copy	"java".toUpperCase() returns "JAVA"
trim()	removes leading/trailing spaces	" hi ".trim() returns "hi"

Figure 3.8 String Methods

String Operations in Code

- `length()` returns the number of characters in the string.
- `charAt(index)` returns the character at the specified zero-based position.
- `toUpperCase()` and `toLowerCase()` return new strings with changed case.
- `equals` compares content; `s1.equals(s2)` is true when both contain the same characters.
- `s1 == s2` is true only when both references point to the same object in memory.
- Always prefer `equals` over `==` when comparing string values in application logic.

Figure 3.9 String Code Example

StringDemo.java

```
public class StringDemo {  
    public static void main(String[] args) {  
        String s1 = "Java";  
        String s2 = new String("Java");  
  
        System.out.println("Length: " + s1.length());  
        System.out.println("Char at 0: " + s1.charAt(0));  
        System.out.println("Upper: " + s1.toUpperCase());  
        System.out.println("Equals s2: " + s1.equals(s2));  
        System.out.println("== s2: " + (s1 == s2)); // false - different objects  
    }  
}
```

Figure 3.9 String Code Example

StringBuilder for Mutable Text

- StringBuilder provides a mutable sequence of characters for efficient text building.
- `append()` adds text to the end without creating a new object for each operation.
- `insert()`, `delete()`, and `reverse()` modify the same internal buffer in place.
- Use StringBuilder inside loops when concatenating many strings to avoid performance overhead.
- Call `toString()` to obtain an immutable String from the StringBuilder when finished.
- StringBuffer is similar but synchronized; StringBuilder is preferred for single-threaded code.

Figure 3.10 StringBuilder Benefits

String Immutability

String (Immutable)

- Once created, content cannot change
- Every modification creates a new String object
- Safe for sharing across threads
- Use for constants and short text

StringBuilder (Mutable)

- Internal buffer can be modified in place
- append(), insert(), delete() change same object
- Efficient for loops and many concatenations
- Not thread-safe unless synchronized

Figure 3.10 StringBuilder Benefits

StringBuilder Code Example

- new StringBuilder() creates an empty builder with default capacity.
- Chained append calls build the sentence without intermediate String objects.
- toString() converts the builder content to a standard String for output or storage.
- insert(0, text) adds characters at the beginning of the buffer.
- reverse() reverses the character sequence in place for demonstration or algorithms.
- StringBuilder significantly reduces memory allocation compared to repeated string + operations.

Figure 3.11 StringBuilder Example

StringBuilderDemo.java

```
public class StringBuilderDemo {  
    public static void main(String[] args) {  
        StringBuilder sb = new StringBuilder();  
  
        sb.append("Object ");  
        sb.append("Oriented ");  
        sb.append("Programming");  
  
        System.out.println(sb.toString());  
        System.out.println("Length: " + sb.length());  
  
        sb.insert(0, "Java - ");  
        sb.reverse();  
        System.out.println(sb);  
    }  
}
```

Figure 3.11 StringBuilder Example

Java I/O Overview

Java I/O Flow

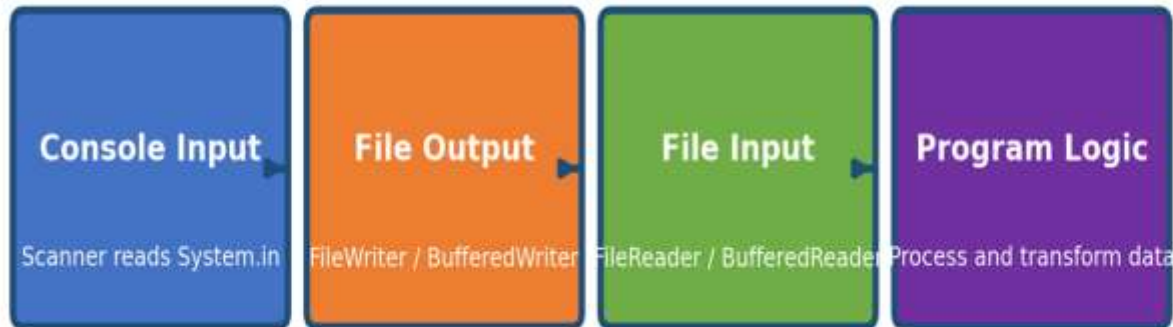


Figure 3.12 Java I/O Flow

Console Input with Scanner

- Scanner reads formatted input from System.in (keyboard) and parses it into types.
- `nextInt()`, `nextDouble()`, and `nextLine()` retrieve different kinds of user input.
- Always display a prompt before reading so the user knows what to enter.
- Menu-driven programs use a loop with switch to repeat actions until the user exits.
- Call `sc.nextLine()` after `nextInt()` to consume the trailing newline character.
- Close the Scanner when the program ends to release the underlying input stream.

Figure 3.13 Console Menu with Scanner

ConsoleMenu.java

```
import java.util.Scanner;

public class ConsoleMenu {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice;

        do {
            System.out.println("1. Add 2. View 3. Exit");
            System.out.print("Choice: ");
            choice = sc.nextInt();
            sc.nextLine(); // consume newline

            switch (choice) {
                case 1: System.out.println("Add selected"); break;
                case 2: System.out.println("View selected"); break;
                case 3: System.out.println("Goodbye!"); break;
                default: System.out.println("Invalid choice");
            }
        } while (choice != 3);

        sc.close();
    }
}
```

Figure 3.13 Console Menu with Scanner

Writing to a File

- FileWriter writes character data to a text file on the filesystem.
- The write method appends strings; include `\n` for line breaks in text files.
- try-with-resources automatically closes the writer even if an exception occurs.
- IOException is checked and must be caught or declared in the method signature.
- If the file does not exist, FileWriter creates it; existing content may be overwritten.
- Always handle I/O errors gracefully with try-catch and informative error messages.

Figure 3.14 FileWriter Example

FileWriteDemo.java

```
import java.io.FileWriter;
import java.io.IOException;

public class FileWriteDemo {
    public static void main(String[] args) {
        try (FileWriter writer = new FileWriter("output.txt")) {
            writer.write("Line 1: Hello from Java\n");
            writer.write("Line 2: OOPJ Semester 3\n");
            System.out.println("File written successfully.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Figure 3.14 FileWriter Example

Reading from a File

- `BufferedReader` wraps `FileReader` to read text files efficiently line by line.
- `readLine()` returns the next line as a `String` or `null` when end of file is reached.
- The `while ((line = reader.readLine()) != null)` pattern processes every line.
- `BufferedReader` reduces disk access by reading data in chunks into an internal buffer.
- `try-with-resources` ensures the reader is closed, preventing resource leaks.
- Combine file reading with string processing methods to parse and analyze file content.

Figure 3.15 FileReader Example

FileReadDemo.java

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class FileReadDemo {
    public static void main(String[] args) {
        try (BufferedReader reader =
            new BufferedReader(new FileReader("output.txt"))) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Figure 3.15 FileReader Example

Integrating Console and File I/O

- Real programs often read user input from the console and persist results to files.
- Read data with Scanner, process it with arrays and strings, then write output with FileWriter.
- Use relative paths (output.txt) for files in the project folder during lab exercises.
- Check that files exist before reading; handle FileNotFoundException in catch blocks.
- Separate I/O code from business logic using methods for readability and testing.
- Mastering basic I/O prepares you for serialization, networking, and database access later.

Figure 3.16 Console and File I/O Integration

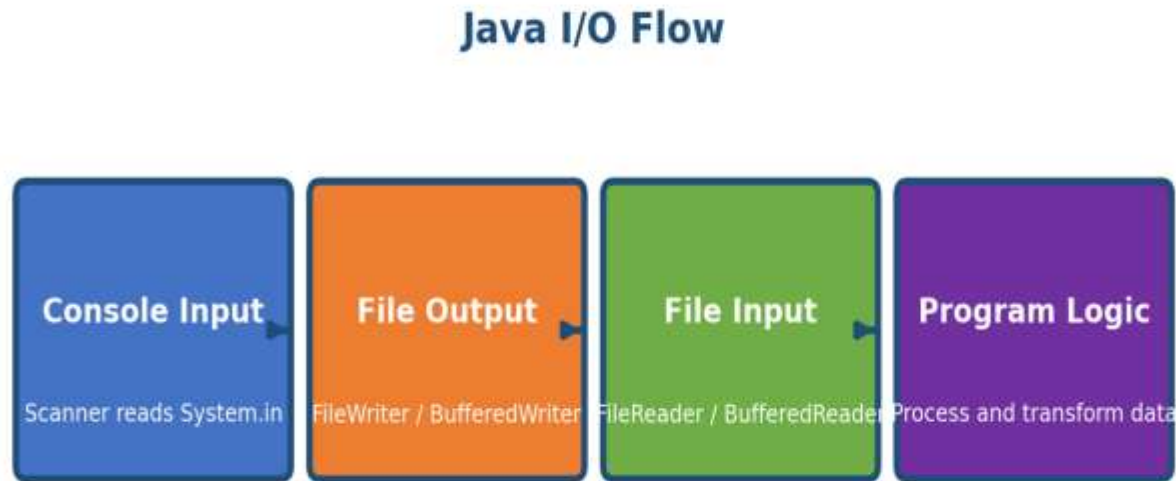


Figure 3.16 Console and File I/O Integration

References

1. Herbert Schildt, Java: The Complete Reference
Cay S. Horstmann, Core Java
Kathy Sierra & Bert Bates, Head First Java

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

